

Business Analytics

Lab Manual

Experiments 3 - 10

- 3. Descriptive Statistics, Correlation Analysis & Predictive Model
 - 4. OLAP Operations (Drill-down, Slice, Dice)
- 5. Data Preprocessing (Missing Values & Normalization)
- 6. Dimensionality Reduction (PCA, KPCA, SVD)
 - 7. Bivariate & Multivariate Analysis
- 8. Classification Model (Customer Churn)
 - 9. Customer Segmentation
- 10. Employee Attrition Prediction

Each experiment includes: Aim, Algorithm, Code, Output, Result

Experiment 3: Descriptive Statistics, Correlation Analysis and Predictive Model

AIM

To perform descriptive statistics (mean, standard deviation, median) and correlation analysis, and build a predictive model using a dataset created as a Pandas DataFrame.

ALGORITHM

- Step 1:** Import required libraries.
- Step 2:** Create a dataset using a Pandas DataFrame.
- Step 3:** Compute descriptive statistics using describe().
- Step 4:** Calculate correlation matrix using corr().
- Step 5:** Split dataset into training and testing data.
- Step 6:** Train a Linear Regression model.
- Step 7:** Predict the output.

CODE

```
# Experiment 3: Descriptive Statistics, Correlation Analysis and Predictive Model
# Aim: To perform descriptive statistics (mean, standard deviation, median) and correlation
analysis,
# and build a predictive model using a dataset created as a Pandas DataFrame.

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Create DataFrame
data = pd.DataFrame({
    'feature1': [10, 20, 30, 40, 50],
    'feature2': [15, 25, 35, 45, 55],
    'target': [100, 200, 300, 400, 500]
})

# Descriptive statistics
print("Descriptive Statistics")
print(data.describe())

# Correlation
print("\nCorrelation Matrix")
print(data.corr())

# Predictive model
X = data[['feature1', 'feature2']]
y = data['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)
print("\nPredicted Value:", pred)
```

OUTPUT

```
Descriptive Statistics
      feature1 feature2 target
count 5.000000 5.000000 5.000000
mean 30.000000 35.000000 300.000000
std 15.811388 15.811388 158.113883
min 10.000000 15.000000 100.000000
25% 20.000000 25.000000 200.000000
50% 30.000000 35.000000 300.000000
75% 40.000000 45.000000 400.000000
max 50.000000 55.000000 500.000000
```

```
Correlation Matrix
      feature1 feature2 target
feature1 1.0 1.0 1.0
feature2 1.0 1.0 1.0
target 1.0 1.0 1.0
```

```
Predicted Value: [400.]
```

RESULT

Thus descriptive statistics, correlation analysis, and predictive modeling were successfully performed using a Pandas DataFrame.

Experiment 4: OLAP Operations using Pandas

AIM

To perform OLAP operations (Drill-down, Slice, Dice) using Pandas DataFrame.

ALGORITHM

Step 1: Create a sales dataset using DataFrame.

Step 2: Perform drill-down using groupby().

Step 3: Perform slice operation using filtering.

Step 4: Perform dice operation using pivot_table().

CODE

```
# Experiment 4: OLAP Operations using Pandas
# Aim: To perform OLAP operations (Drill-down, Slice, Dice) using Pandas DataFrame.

import pandas as pd

data = pd.DataFrame({
    'Region': ['North', 'South', 'North', 'East', 'South'],
    'Product': ['A', 'B', 'A', 'B', 'A'],
    'Sales': [100, 200, 150, 300, 250]
})

# Drill Down
print("Drill Down")
print(data.groupby('Region')['Sales'].sum())

# Slice
print("\nSlice (South Region)")
print(data[data['Region']=="South"])

# Dice
pivot = pd.pivot_table(data, values='Sales',
                        index='Region',
                        columns='Product',
                        aggfunc='sum')

print("\nDice Operation")
print(pivot)
```

OUTPUT

```
Drill Down
Region
East 300
North 250
South 450
Name: Sales, dtype: int64

Slice (South Region)
  Region Product Sales
1  South B      200
4  South A      250

Dice Operation
Product A B
Region
East NaN 300.0
North 250.0 NaN
South 250.0 200.0
```

RESULT

Thus OLAP operations such as Drill-down, Slice, and Dice were successfully simulated using Pandas DataFrame.

Experiment 5: Data Preprocessing (Missing Values & Normalization)

AIM

To perform data preprocessing operations including handling missing values and normalization.

ALGORITHM

Step 1: Create DataFrame containing missing values.

Step 2: Replace missing values with mean.

Step 3: Normalize data using MinMaxScaler.

CODE

```
# Experiment 5: Data Preprocessing (Missing Values & Normalization)
# Aim: To perform data preprocessing operations including handling missing values and
normalization.

import pandas as pd
from sklearn.preprocessing import MinMaxScaler

data = pd.DataFrame({
    'Age': [22, 25, None, 28, 30],
    'Salary': [25000, 30000, 32000, None, 40000]
})

# Handle missing values
data = data.fillna(data.mean())

print("After Handling Missing Values")
print(data)

# Normalization
scaler = MinMaxScaler()
scaled = scaler.fit_transform(data)

print("\nNormalized Data")
print(scaled)
```

OUTPUT

```
After Handling Missing Values
   Age  Salary
0  22.00  25000.0
1  25.00  30000.0
2  26.25  32000.0
3  28.00  31750.0
4  30.00  40000.0

Normalized Data
[[0.  0. ]
 [0.375 0.33333333]
 [0.53125 0.46666667]
 [0.75 0.45 ]
 [1. 1. ]]
```

RESULT

Thus missing values were handled and the dataset was normalized successfully.

Experiment 6: Dimensionality Reduction using PCA, KPCA and SVD

AIM

To perform dimensionality reduction using PCA, Kernel PCA, and SVD.

ALGORITHM

Step 1: Create dataset using DataFrame.

Step 2: Apply PCA.

Step 3: Apply Kernel PCA.

Step 4: Apply SVD.

Step 5: Display reduced features.

CODE

```
# Experiment 6: Dimensionality Reduction using PCA, KPCA and SVD
# Aim: To perform dimensionality reduction using PCA, Kernel PCA, and SVD.

import pandas as pd
from sklearn.decomposition import PCA, KernelPCA, TruncatedSVD

data = pd.DataFrame({
    'x1': [1, 2, 3, 4, 5],
    'x2': [2, 3, 4, 5, 6],
    'x3': [5, 4, 3, 2, 1]
})

# PCA
pca = PCA(n_components=2)
pca_result = pca.fit_transform(data)

# KPCA
kpca = KernelPCA(n_components=2, kernel='rbf')
kpca_result = kpca.fit_transform(data)

# SVD
svd = TruncatedSVD(n_components=2)
svd_result = svd.fit_transform(data)

print("PCA Result\n",pca_result)
print("\nKPCA Result\n",kpca_result)
print("\nSVD Result\n",svd_result)
```

OUTPUT

PCA Result

```
[[ -3.46410162e+00 -3.43990023e-16]
 [ -1.73205081e+00  1.14663341e-16]
 [  0.00000000e+00  0.00000000e+00]
 [  1.73205081e+00 -1.14663341e-16]
 [  3.46410162e+00 -2.29326682e-16]]
```

KPCA Result

```
[[ 5.90028805e-01 -4.94107735e-01]
 [ 5.75518974e-01  1.44621826e-01]
 [ 5.17647299e-16  6.98971818e-01]
 [-5.75518974e-01  1.44621826e-01]
 [-5.90028805e-01 -4.94107735e-01]]
```

SVD Result

```
[[ 4.21681372  3.49549454]
 [ 5.01622223  1.95895751]
 [ 5.81563074  0.42242048]
 [ 6.61503925 -1.11411654]
 [ 7.41444777 -2.65065357]]
```

RESULT

Thus dimensionality reduction was performed using PCA, KPCA, and SVD successfully.

Experiment 7: Bivariate and Multivariate Analysis

AIM

To perform bivariate and multivariate analysis using visualization techniques.

ALGORITHM

Step 1: Create dataset using DataFrame.

Step 2: Plot scatter plot for bivariate analysis.

Step 3: Plot pairplot for multivariate analysis.

CODE

```
# Experiment 7: Bivariate and Multivariate Analysis
# Aim: To perform bivariate and multivariate analysis using visualization techniques.

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [5, 4, 3, 2, 1],
    'C': [2, 3, 4, 5, 6]
})

# Bivariate
sns.scatterplot(x='A', y='B', data=data)
plt.show()

# Multivariate
sns.pairplot(data)
plt.show()
```

OUTPUT

RESULT

Thus bivariate and multivariate analysis was successfully performed.

Experiment 8: Classification Model (Customer Churn)

AIM

To build a classification model using Logistic Regression.

ALGORITHM

- Step 1:** Create dataset using DataFrame.
- Step 2:** Split dataset into training and testing sets.
- Step 3:** Train logistic regression model.
- Step 4:** Predict churn values.

CODE

```
# Experiment 8: Classification Model (Customer Churn)
# Aim: To build a classification model using Logistic Regression.

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

data = pd.DataFrame({
    'Usage': [10, 20, 30, 40, 50],
    'Cost': [100, 200, 300, 400, 500],
    'Churn': [0, 0, 1, 1, 1]
})

X = data[['Usage', 'Cost']]
y = data['Churn']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = LogisticRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Prediction:", pred)
```

OUTPUT

Prediction: [0]

RESULT

Thus a Logistic Regression classification model was successfully implemented.

Experiment 9: Customer Segmentation

AIM

To segment customers based on purchase behavior using K-Means clustering.

ALGORITHM

Step 1: Create customer dataset.

Step 2: Select features.

Step 3: Apply KMeans clustering.

Step 4: Assign clusters.

CODE

```
# Experiment 9: Customer Segmentation
# Aim: To segment customers based on purchase behavior using K-Means clustering.

import pandas as pd
from sklearn.cluster import KMeans

data = pd.DataFrame({
    'Income': [15, 16, 17, 40, 42, 44],
    'Spending': [39, 40, 41, 70, 72, 74]
})

kmeans = KMeans(n_clusters=2)
data['Cluster'] = kmeans.fit_predict(data)

print(data)
```

OUTPUT

```
   Income  Spending  Cluster
0     15       39         0
1     16       40         0
2     17       41         0
3     40       70         1
4     42       72         1
5     44       74         1
```

RESULT

Thus customers were successfully segmented using K-Means clustering.

Experiment 10: Employee Attrition Prediction

AIM

To predict whether an employee will leave the company using Decision Tree classification.

ALGORITHM

Step 1: Create employee dataset.

Step 2: Split dataset.

Step 3: Train Decision Tree model.

Step 4: Predict employee attrition.

CODE

```
# Experiment 10: Employee Attrition Prediction
# Aim: To predict whether an employee will leave the company using Decision Tree classification.

import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

data = pd.DataFrame({
    'Age': [25, 30, 35, 40, 45],
    'Salary': [30000, 40000, 50000, 60000, 70000],
    'Attrition': [0, 0, 1, 1, 1]
})

X = data[['Age', 'Salary']]
y = data['Attrition']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = DecisionTreeClassifier()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Prediction:", pred)
```

OUTPUT

```
Prediction: [0]
```

RESULT

Thus employee attrition prediction was successfully implemented using a Decision Tree classifier.